SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Harshavardhan RV	**Reg. No.:**	192472163
Section / Batch:	CSE AI	**Date:**	16/07/2026

Code of Conduct

*I **Harshavardhan RV(192472163)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Harshavardhan RV**

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

NAME: Sodisetty Sahithya **REG NO:** 192472186

COURSE CODE: DSA0307 Natural Language Processing

1)A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

AIM:

To develop a Python program using Regular Expressions to automatically extract candidate information from resumes and identify eligible candidates.

ALGORITHM:

Import the re module.

Read the resume text.

Extract the candidate name.

Extract email address.

Extract mobile number.

Search for technical skills.

Extract years of experience.

Display candidate profile.

Check eligibility:

Experience ≥ 2 years

Python skill available

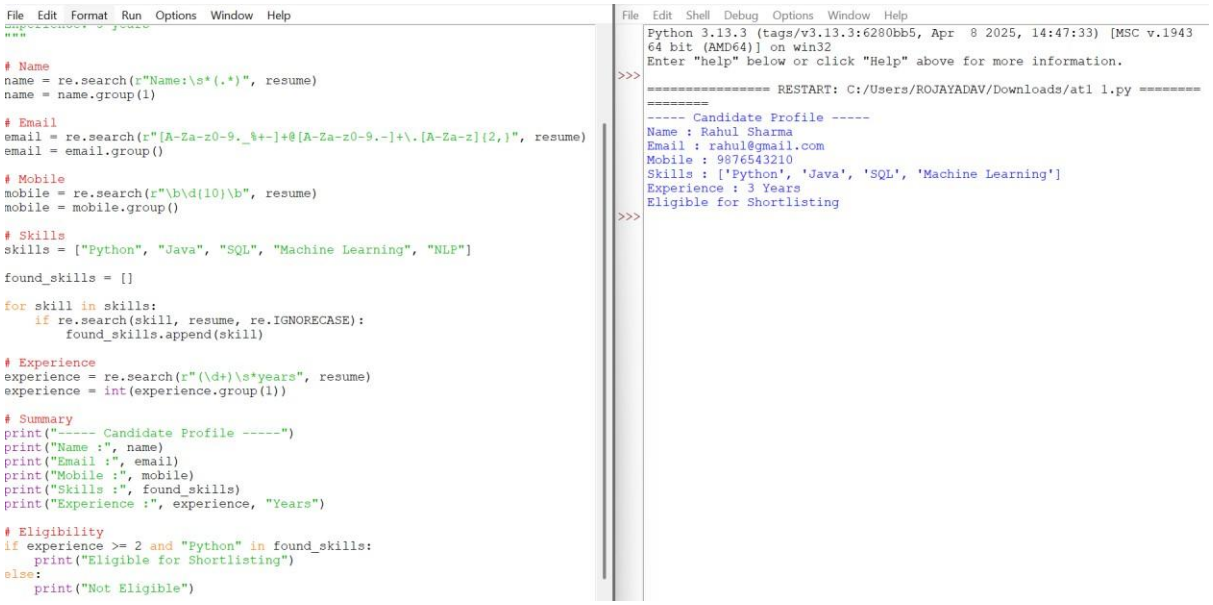
Display eligibility result.

CODE:

```
import re
resume = """
Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years

# Name
name = re.search(r"Name:\s*(.*)", resume)
name = name.group(1)

# Email
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
email = email.group()

# Mobile
mobile = re.search(r"\b\d{10}\b", resume)
mobile = mobile.group()

# Skills
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

found_skills = []

for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        found_skills.append(skill)
```

```
# Experience
experience = re.search(r"(\d+)\s*years", resume)
experience = int(experience.group(1))

# Summary
print("----- Candidate Profile------")
print("Name :", name)
print("Email :", email)
print("Mobile :", mobile)
print("Skills :", found_skills)
print("Experience :", experience, "Years")

# Eligibility
if experience >= 2 and "Python" in found_skills:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")
```

OUTPUT:



RESULT: The Program Was Executed Successfully.

2)An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

AIM:

To develop a product search system using Regular Expressions for exact, prefix, suffix, partial, and case-insensitive searches.

ALGORITHM:

Import re.

Create a product list.

Accept search keyword.

Perform:

Exact search

Prefix search

Suffix search

Partial search

Case-insensitive search

Display matching products.

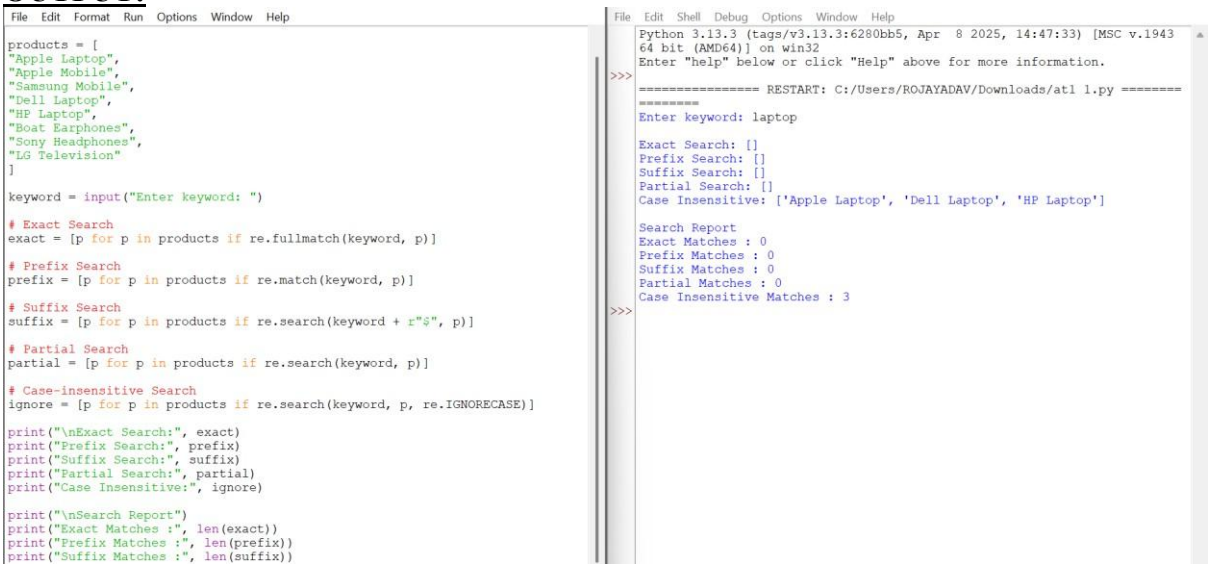
Display total matches.

CODE:

```
import re

products = [
"Apple Laptop",
"Apple Mobile",
"Samsung Mobile",
"Dell Laptop",
"HP Laptop",
"Boat Earphones",
"Sony Headphones",
"LG Television"
]
keyword = input("Enter keyword: ")

# Exact Search
exact = [p for p in products if re.fullmatch(keyword, p)]

# Prefix Search
```

```
prefix = [p for p in products if re.match(keyword, p)]

# Suffix Search
suffix = [p for p in products if re.search(keyword + r"$", p)]

# Partial Search
partial = [p for p in products if re.search(keyword, p)]

# Case-insensitive Search
ignore = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

print("\nExact Search:", exact)
print("Prefix Search:", prefix)
print("Suffix Search:", suffix)
print("Partial Search:", partial)
print("Case Insensitive:", ignore)

print("\nSearch Report")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
print("Suffix Matches :", len(suffix))
print("Partial Matches :", len(partial))
print("Case Insensitive Matches :", len(ignore))
```

OUTPUT:



RESULT: The Program Was Executed Successfully.

3)A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field successful.

AIM:

To validate student registration details using Regular Expressions.

ALGORITHM:

Import re.

Read:

Register Number

Email

Course Code

Semester

Mobile Number

Validate each field.

Display validation result.

If all validations pass, display Registration Successful.

CODE:

```
import re

reg = input("Register Number: ")
email = input("Institution Email: ")
course = input("Course Code: ")
semester = input("Semester: ")
mobile = input("Mobile Number: ")

valid = True

# Register Number
if re.fullmatch(r"\d{12}", reg):
    print("Register Number Valid")
else:
    print("Invalid Register Number")
    valid = False
```

```
# Email
if re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email):
    print("Email Valid")
else:
    print("Invalid Email")
    valid = False

# Course Code
if re.fullmatch(r"[A-Z]{3}\d{2}", course):
    print("Course Code Valid")
else:
    print("Invalid Course Code")
    valid = False

# Semester
if re.fullmatch(r"[1-8]", semester):
    print("Semester Valid")
else:
    print("Invalid Semester")
    valid = False

# Mobile
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number Valid")
else:
    print("Invalid Mobile Number")
    valid = False

if valid:
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")
```

OUTPUT:

```
File  Edit  Format  Run  Options  Window  Help
valid = True

# Register Number
if re.fullmatch(r"\d{12}", reg):
    print("Register Number Valid")
else:
    print("Invalid Register Number")
    valid = False

# Email
if re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email):
    print("Email Valid")
else:
    print("Invalid Email")
    valid = False

# Course Code
if re.fullmatch(r"[A-Z]{3}\d{2}", course):
    print("Course Code Valid")
else:
    print("Invalid Course Code")
    valid = False

# Semester
if re.fullmatch(r"[1-8]", semester):
    print("Semester Valid")
else:
    print("Invalid Semester")
    valid = False

# Mobile
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number Valid")
else:
    print("Invalid Mobile Number")
    valid = False

if valid:
```

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ROJAYADAV/Downloads/at1 1.py ========
========
Register Number: 192424226
Institution Email: roja226@gmail.com
Course Code: DSA0305
Semester: 3
Mobile Number: 9876543210
Invalid Register Number
Invalid Email
Invalid Course Code
Semester Valid
Mobile Number Valid

Registration Failed
>>>
========== RESTART: C:/Users/ROJAYADAV/Downloads/at1 1.py ========
========
Register Number: 1923456789012
Institution Email: roja@saveetha.com
Course Code: DSA03
Semester: 3
Mobile Number: 9876543210
Invalid Register Number
Email Valid
Course Code Valid
Semester Valid
Mobile Number Valid

Registration Failed
>>>
```

RESULT: The Program Was Executed Successfully.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____